5. Technology Comparison

Method: for each key decision, 2–3 alternatives are scored 1–5 against weighted criteria derived from the requirements (doc 06). Points = score × weight. The best total is chosen unless a hard constraint excludes it. Scores are the team's judgement from documentation and experience and should be reviewed with the supervisor; AI choices are finally decided by the experiments in doc 09.

TD1. Backend platform

Criterion	Weight	FastAPI (Python)	Django REST Framework	Node.js (NestJS)
Same language as the AI modules (Python)	5	5	5	2
Development speed for a 3-person team	4	4	4	3
Automatic API documentation (OpenAPI)	3	5	3	3
Built-in admin/auth	3	2	5	2
Team familiarity *(adjust)*	3	3	3	3
Total		**25+16+15+6+9 = 71**	25+16+9+15+9 = 74	10+12+9+6+9 = 46

Django scores slightly higher in this draft because of its built-in admin and auth. **Decision: Python REST API** (the proposal fixes "a Python REST API"); choose Django REST Framework if the team prefers batteries included, FastAPI if it prefers lightweight async and generated docs. *Record the final choice with the supervisor.*

TD2. Spatial data storage

Criterion	Weight	PostgreSQL + PostGIS	MongoDB	MySQL (spatial)
Spatial types and functions	5	5	3	3
Relational integrity (reservations, users, reports)	4	5	2	5
Transactions for capacity checks in reservation	4	5	3	4
GeoJSON output	3	5	4	3
Tooling/ORM support	3	5	4	4
Total		**25+20+20+15+15 = 95**	15+8+12+12+12 = 59	15+20+16+9+12 = 72

Decision: PostgreSQL + PostGIS (also in the proposal).

TD3. Map client library (OpenStreetMap-based)

Criterion	Weight	Leaflet	MapLibre GL JS	OpenLayers
Speed of implementation	5	5	3	2
GeoJSON layers, popups, filters, drawing	4	5	4	5
Heatmap / clustering for staff map	3	4	5	4
No paid token	4	5	5	5
Community/examples	2	5	4	3
Total		**25+20+12+20+10 = 87**	15+16+15+20+8 = 74	10+20+12+20+6 = 68

Decision: Leaflet (MapLibre as upgrade path for large heatmaps).

TD4. Hazard-report text classification (KZ/RU/EN)

Criterion	Weight	Multilingual pretrained encoder + simple classifier	TF-IDF + logistic regression	Large language model API (zero-shot)
Handles Kazakh, Russian, English	5	4	2	4
Works with ~600 labelled reports	4	4	4	5
Local, reproducible, no external data transfer (privacy)	4	5	5	1
Explainability / confidence output	3	4	5	2
Compute cost for a student project	2	3	5	3
Total		**20+16+20+12+6 = 74**	10+16+20+15+10 = 71	20+20+4+6+6 = 56

Decision: multilingual pretrained text encoder with a simple classifier (as in the proposal); TF-IDF + logistic regression is kept as a second baseline next to keyword rules because it is close in score.

TD5. Lost-item matching

Criterion	Weight	Pretrained image embeddings + text + location similarity	Perceptual hashing / color histograms	Train a dedicated matching network (like [20])
Works without large training data	5	5	5	2
Matching quality for different photos of one item	5	4	2	5
Uses location and time, not only image	3	5	3	3

Criterion	Weight	Pretrained image embeddings + text + location similarity	Perceptual hashing / color histograms	Train a dedicated matching network (like [20])
Implementation effort	3	4	5	2
Total		**25+20+15+12 = 72**	25+10+9+15 = 59	10+25+9+6 = 50

Decision: pretrained embeddings combined with text and location similarity (proposal), evaluated by top-5 retrieval against a text filter.

TD6. Parking occupancy prediction (30 min)

Criterion	Weight	Time-series regression (e.g. gradient boosting on lag/time features)	LSTM / sequence network	Seasonal / ARIMA statistical model
Accuracy on daily/weekly patterns	4	4	4	3
Little data / fast training	4	5	2	5
Explainability	3	4	2	5
Simple deployment in the Python API	3	5	3	4
Total		**16+20+12+15 = 63**	16+8+6+9 = 39	12+20+15+12 = 59

Decision: time-series regression (proposal); baseline = historical average.

TD7. Component interaction

Criterion	Weight	REST/JSON over HTTPS	WebSocket push	Message broker (MQTT)
Simplicity, testability	5	5	3	2
Fit with update rates (minutes)	4	5	4	4
Standard tooling	3	5	2	3
Total		**25+20+15 = 60**	15+16+6 = 37	10+16+9 = 35

Decision: REST/JSON. AI modules run as modules/services behind the REST API.

Summary of the stack

Layer	Choice
Frontend	Web map interface (HTML/JS or React) + Leaflet + OpenStreetMap tiles
Backend	Python REST API (FastAPI or Django REST Framework — to confirm)
AI	Python; multilingual text encoder + classifier; image embeddings + similarity; time-series regression
Database	PostgreSQL + PostGIS
Data	Synthetic generators; OpenStreetMap; open data if available
DevOps	Git, Docker Compose, YouTrack

Risks

Risk	Mitigation
Pretrained model licences/size	Check licences; use smaller models; document
OSM public tile server not for heavy use	Use only for development, or a tile provider
Weak Kazakh performance of the encoder	Report per language; add data augmentation